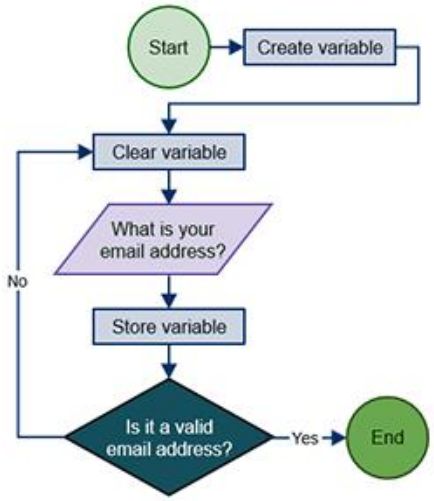
Data Structures and Algorithms



Lecture 10

Algorithms and Sorting techniques

Dept. of ICT, Faculty of Technology- UoR, 2026

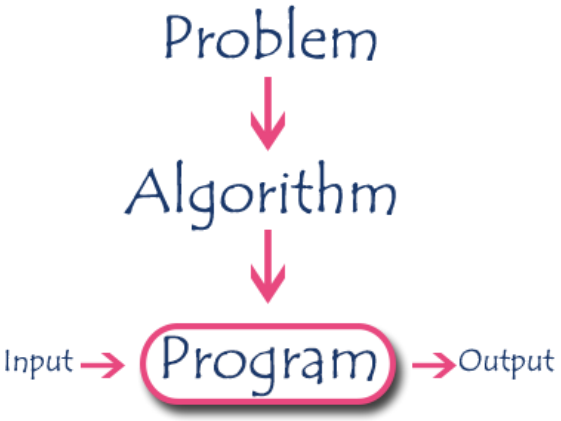
Learning Objectives

- Define a computer algorithm, properties of algorithm and performance analysis of the algorithm.
- Explain the concept of: **time complexity** and **space complexity.**
- Discuss on types of **asymptotic notations** such as **Big Oh** notation, **Big Omega** notation etc.
- Explain on **sorting techniques**.

What is an algorithm?

- An algorithm is a step-by-step procedure (sequence of statements) to solve a problem.
- Algorithms are used to convert our problem solution into step-by-step statements.
- These statements can be converted into computer programming instructions which forms a program.
- This program is executed by computer to produce solution.

Program takes required data as input, processes data according to the program instructions and finally produces result.



Properties of the algorithms

Algorithm must satisfy the following properties:

1. **Input**- There should be 0 or more inputs supplied externally to the algorithm.

2. **Output**- There should be at least 1 output obtained.

3. **Definiteness**- Every step of the algorithm should be clear and well defined.

4. **Finiteness**- The algorithm should have finite number of steps.

5. **Correctness**- Every step of the algorithm must generate a correct output.

Performance Analysis of an algorithm

- When we have more than one algorithm to solve a problem, we need to select the best one. **Performance analysis** helps us to select the best algorithm from multiple algorithms to solve a problem.

Performance of an algorithm depends on the following elements...

1. Whether that algorithm is providing the exact solution for the problem?
2. Whether it is easy to understand?
3. Whether it is easy to implement?
4. How much space (memory) it requires to solve the problem?
5. How much time it takes to solve the problem? Etc.,

Performance Analysis of an algorithm

- Performance analysis of an algorithm is
  - **Process of calculating space required by that algorithm and**
  - **Time required by that algorithm.**

Performance analysis of an algorithm is performed by using the following measures...

1.Space required to complete the task of that algorithm (**Space Complexity**). It includes program space and data space.

2.Time required to complete the task of that algorithm (**Time Complexity**)

Space Complexity

When we design an algorithm to solve a problem, it needs some computer memory to complete its execution. For any algorithm, memory is required for the following purposes.

1.Memory required to store program instructions

2.Memory required to store constant values

3.Memory required to store variable values

And for few other things

Total amount of computer memory required by an algorithm to complete its execution is called as **space complexity** of that algorithm

Space Complexity

To calculate the space complexity, user must know the memory required to store different data type values (according to the compiler).

Example: C Programming Language compiler requires the following.

1. 2 bytes to store Integer value,
2. 4 bytes to store Floating Point value,
3. 1 byte to store Character value,
4. 6 (OR) 8 bytes to store double value

Space Complexity

Consider the following piece of code

```
int cal (int b)
{
        return b*b;
}
```

- In above piece of code, it requires 2 bytes of memory to store variable ‘b' and another 2 bytes of memory is used for return value.
- Totally it requires 4 bytes of memory to complete its execution.
- And this 4 bytes of memory is fixed for any input value of ‘b'.
- This space complexity is said to be **Constant Space Complexi**ty.

Time Complexity

Every algorithm requires some amount of computer time to execute its instruction to perform the task. This computer time required is called **time complexity**.

Running time of an algorithm depends upon the following.

1.Whether it is running on **Single** processor machine or **Multi** processor machine.

2.Whether it is a **32 bit** machine or **64 bit** machine

3.Read and Write speed of the machine.

4.The time it takes to perform **Arithmetic** operations, **logical** operations, **return** value and **assignment** operations etc.,

5.Input data

If any program requires fixed amount of time for all input values then its time complexity is said to be **Constant Time Complexity**.

Time Complexity

- A good software engineer will consider **time complexity** when planning their program. From the start, an engineer should consider a scenario that their program may encounter that would require the most time to complete if at all.

- Time Complexity is most commonly estimated by counting the number of elementary steps performed by any algorithm to finish execution.

- Like in the example of previous slide sample code, for the first code the loop will run n number of times, so the time complexity will be n at least and as the value of n will increase the time taken will also increase.

Since the algorithm's performance may vary with different types of input data, hence for an algorithm we usually use the **worst-case time complexity** of an algorithm because that is the maximum time taken for any input size.

Asymptotic Notations

- When it comes to analyzing the complexity of any algorithm in terms of time and space, we can never provide an exact number to define the time required and the space required by the algorithm, instead we express it using some standard notations, also known as **Asymptotic Notations**.

- In asymptotic notation, when we want to represent the complexity of an algorithm, we use only the **most significant terms** in the complexity of that algorithm and ignore least significant terms in the complexity of that algorithm (Here complexity may be Space Complexity or Time Complexity).

asymptotic

An asymptotic line is a line that gets closer and closer to a curve as the distance gets closer to infinity.

Asymptotic Notations

Example:

consider the following time complexities of two algorithms.

Algorihtm 1 : $5n^2 + 2n + 1$

Algorihtm 2 : $10n^2 + 8n + 3$

- Here for larger value of 'n' the value of most significant terms ($5n^2$ and $10n^2$) is very larger than the value of least significant terms ($2n + 1$ and $8n + 3$).
- So for larger value of 'n' we ignore the least significant terms to represent overall time required by an algorithm.
- In asymptotic notation, we use only the most significant terms to represent the time complexity of an algorithm.

Types of Asymptotic Notations

1. Big - Oh (O)
2. Big - Omega (Ω)
3. Big - Theta (Θ)

Big - Oh Notation (O)

Big - Oh notation is used to define the **upper bound** of an algorithm in terms of Time Complexity.

Big - Oh notation always indicates the maximum time required by an algorithm for all input values. That means Big - Oh notation describes the **worst case of an algorithm time complexity**.

Consider function **f(n)** the time complexity of an algorithm and **g(n)** is the most significant term. If **f(n) <= C g(n)** for all **n >= n_0, C > 0** and **n_0 >= 1**. Then we can represent **f(n)** as **O(g(n))**.

f(n) = O(g(n))

Big Oh used to describe the effect of the input size on an algorithm performance (running time or memory usage)

Common Big O values

$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$	Exponential

Big - Oh Notation (O)

- It is generally written as $O(f(x_1, x_2, \ldots, x_n))$

- Polynomial time algorithms,
  - $O(1)$ --- Constant time --- the time does not change in response to the size of the problem.
  - $O(n)$ --- Linear time --- the time grows linearly with the size (n) of the problem.
  - $O(n^2)$ --- Quadratic time --- the time grows quadratically with the size (n) of the problem. In big O notation, all polynomials with the same degree are equivalent, so $O(3n^2 + 3n + 7) = O(n^2)$
- Sub-linear time algorithms
  - $O(\log n)$ -- Logarithmic time

Big ‘O’ Notation

Algorithm	**Running Time in Big O Notation**
Linear Search	O(N)
Binary Search	O(log N)
Insertion in unordered array	O(1)
Insertion in ordered array	O(N)
Deletion in unordered array	O(N)
Deletion in ordered array	O(N)

Big O examples

O(n)

▪For example, *O*(*n*) means that an algorithm has a linear complexity. In other words, it **takes ten times longer to operate on a set of 100 items than it does on a set of 10** items (10 * 10 = 100).

```
void printAllElementOfArray(int arr[], int size)
{
   for (int i = 0; i < size; i++)
   {
      printf("%d\n", arr[i]);
   }
}
```

This function runs in O(n) time (or "linear time"), where n is the number of items in the array. If the array has 10 items, we have to print 10 times. If it has 1000 items, we have to print 1000 times.

Big O examples ….Cont..

O(1)

```
void printFirstElementOfArray(int arr[])
{
   printf("First element of array = %d",arr[0]);
}
```

This function runs in O(1) time (or "constant time") relative to its input. The input array could be 1 item or 1,000 items, but this function would still just require one step.

Big O examples ….Cont..

$O(n^2)$

- If the complexity was $O(n^2)$ (quadratic complexity), then it would **take 100 times longer to operate on a set of 100 items than it does on a set of 10 items**.

```
void printAllPossibleOrderedPairs(int arr[], int size)
{
   for (int i = 0; i < 5; i++)
   {
      for (int j = 0; j < 4; j++)
      {
         printf("%d = %d\n", arr[i], arr[j]);
      }
    }
}
```

Here we're nesting two loops. If our array has n items, our outer loop runs n times and our inner loop runs n times for each iteration of the outer loop, giving us n^2 total prints. Thus this function runs in $O(n^2)$ time (or "quadratic time"). If the array has 10 items, we have to print 100 times. If it has 1000 items, we have to print 1000000 times.

Algorithm Analysis

- How to predict an algorithm's performance?
- How well an algorithm scales up?
- How to compare different algorithms for a problem?

Example: Selection Problem

- Given a list of N numbers, determine the *k*th **largest,** where $k \leq N$.
- Algorithm 1:
  (1) Read N numbers into an array
  (2) Sort the array in decreasing order by some simple algorithm
  (3) Return the element in position k

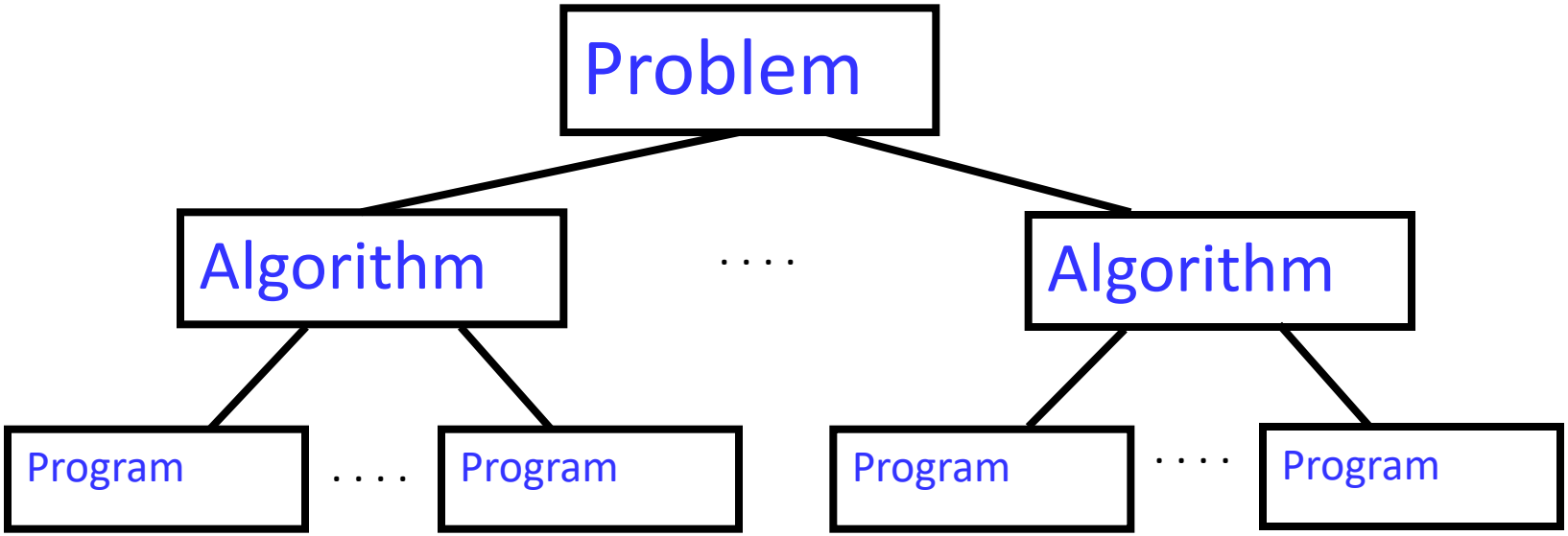
Example: Selection Problem…

- Algorithm 2:
  (1) Read the first k elements into an array and sort them in decreasing order
  (2) Each remaining element is read one by one
    - If smaller than the kth element, then it is ignored
    - Otherwise, it is placed in its correct spot in the array, bumping one element out of the array.
  (3) The element in the kth position is returned as the answer.

Example: Selection Problem…

- Which algorithm is better when
  - N =100 and k = 100?
  - N =100 and k = 1?
- What happens when N = 1,000,000 and k = 500,000?
- There exist better algorithms

Relations between Problems, Algorithms, and Programs



Algorithm Analysis

- We only analyze *correct* algorithms
- An algorithm is correct
  - If, for every input instance, it halts with the correct output
- Incorrect algorithms
  - Might not halt at all on some input instances
  - Might halt with other than the desired answer
- Analyzing an algorithm
  - Predicting the resources that the algorithm requires
  - Resources include
    - Memory
    - Communication bandwidth
    - Computational time (usually most important)

Algorithm Analysis…cont

- Factors affecting the running time
  - computer
  - compiler
  - algorithm used
  - input to the algorithm
    - The content of the input affects the running time
    - typically, the *input size* (number of items in the input) is the main consideration
      - E.g. sorting problem $\Rightarrow$ the number of items to be sorted
      - E.g. multiply two matrices together $\Rightarrow$ the total number of elements in the two matrices
- Machine model assumed
  - Instructions are executed one after another, with no concurrent operations $\Rightarrow$ Not parallel computers

Example

- Calculate

```
int sum(int n)
{
      int partialSum;

1     partialSum=0;                    1
2     for (int i=1;i<=n;i++)           2N+2
3        partialSum += i*i*i;          4N
4     return partialSum;               1
}
```

- Lines 1 and 4 count for one unit each
- Line 3: executed N times, each time four units
- Line 2: (1 for initialization, N+1 for all the tests, N for all the increments) total 2N + 2
- total cost: 6N + 4 $\Rightarrow$ O(N)

Worst- / average- / best-case

- **Worst-case running time of an algorithm**
  - The longest running time for **any** input of size n
  - An upper bound on the running time for any input
    $\Rightarrow$ guarantee that the algorithm will never take longer
  - Example: Sort a set of numbers in increasing order; and the data is in decreasing order
  - The worst case can occur fairly often
    - E.g. in searching a database for a particular piece of information
- Best-case running time
  - sort a set of numbers in increasing order; and the data is already in increasing order
- Average-case running time
  - May be difficult to define what “average” means

How to measure algorithm performance?

What metric should be used to judge algorithms?

- Length of the program (lines of code)
- Ease of programming (bugs, maintenance)
- Memory required
- Running time

Running time is the dominant standard.

- Quantifiable and easy to compare

Analyzing an Algorithm

Simple statement sequence s1; s2; …. ; sk

- Time Complexity is O(1)

ex: accessing any single element in an array takes constant time as only one operation has to be performed

- Simple loops for(i=0;i<n;i++) { s; }

where s is O(1)

Time complexity is O(n)

Analyzing Algorithms

- Nested loops

```
for(i=0;i<n;i++)
 for(j=0;j<n;j++) { s; }
```

Complexity is *n O(n)* or $O(n^2)$

- Loop index depends on outer loop index

```
for(j=0;j<n;j++)
for(k=0;k<j;k++){ s; }
```

Complexity $O(n^2)$

Sorting Techniques

Sorting Techniques

- Sorting refers to arranging data in a particular format.
- Sorting algorithm specifies the way to arrange data in a particular order.
- Most common orders are in numerical or lexicographical order.
- The importance of sorting lies in the fact that data searching can be optimized to a very high level, if data is stored in a sorted manner.
- Sorting is also used to represent data in more readable formats.

Sorting Techniques

Selection sort

Recursive algorithms

Towers of Hanoi

Merge sort

Quick sort

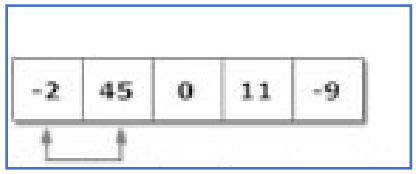
Bubble sort

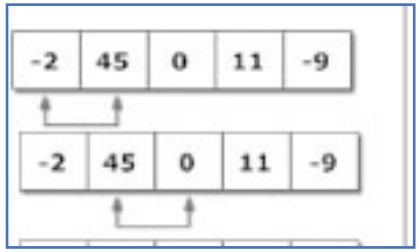
Insertion sort

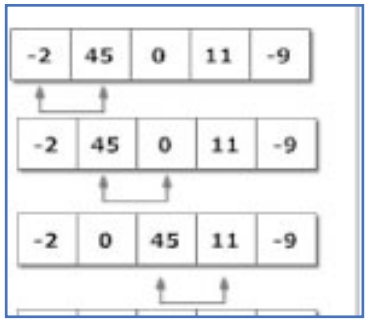
and many more….

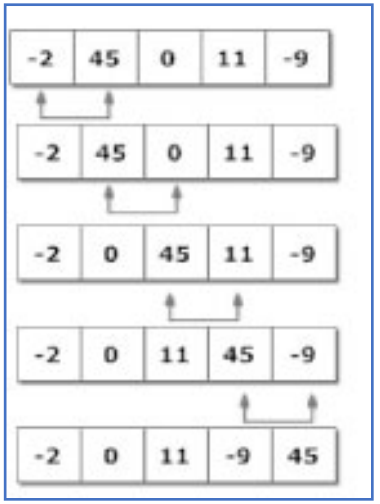
Bubble sort

- Bubble sort is a simple sorting algorithm that works by repeatedly stepping through the list to be sorted.
- Bubble sort algorithm starts by comparing the first two elements of an array and swapping if necessary.









Example:

if you want to sort the elements of array in ascending order and if the first element is greater than second then, you need to swap the elements but, if the first element is smaller than second, you mustn't swap the element.

(5 1 4 2 8) ⟶ (1 5 4 2 8), Here, compares the first two elements, and swaps since 5 > 1.

(1 5 4 2 8) ⟶ (1 4 5 2 8), Swap since 5 > 4

Bubble sort

Bubble sort pass through the list is repeated until no swaps are needed.

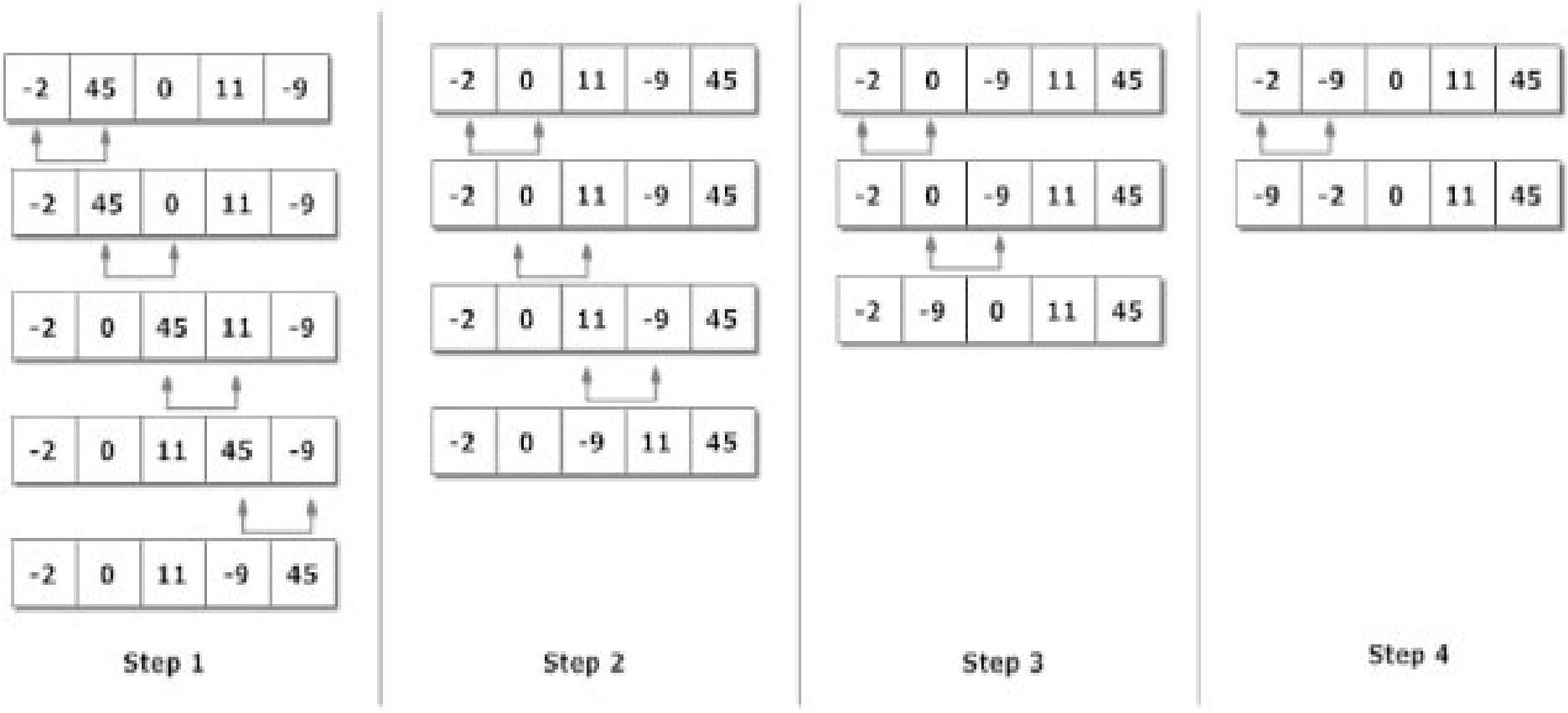


Figure: Working of Bubble sort algorithm

Source: https://www.programiz.com/dsa/bubble-sort

Bubble sort

Other example

First Pass:

(5 1 4 2 8) ⟶ (1 5 4 2 8), compares the first two elements, and swaps since 5 > 1.
(1 5 4 2 8) ⟶ (1 4 5 2 8), Swap since 5 > 4
(1 4 5 2 8) ⟶ (1 4 2 5 8), Swap since 5 > 2
(1 4 2 5 8) ⟶ (1 4 2 5 8), Now, since these elements are already in order (8 > 5), algorithm does not swap them.

Second Pass:

(1 4 2 5 8) ⟶ (1 4 2 5 8)
(1 4 2 5 8) ⟶ (1 2 4 5 8), Swap since 4 > 2
(1 2 4 5 8) ⟶ (1 2 4 5 8)
(1 2 4 5 8) ⟶ (1 2 4 5 8)

Now, the array is already sorted, but our algorithm does not know if it is completed. The algorithm needs one whole pass without any swap to know it is sorted.

Bubble sort

Algorithm

Step 1: Repeat Steps 2 and 3 for i=1 to 10

Step 2: Set j=1

Step 3: Repeat while j<=n

(A) if a[i] < a[j]

Then interchange a[i] and a[j]

[End of if]

(B) Set j = j+1

[End of Inner Loop]

[End of Step 1 Outer Loop]

Step 4: Exit

Bubble sort

C program

```
#include<stdio.h>
void main()
{
 int i,n,temp,j,arr[25];

printf("Enter the number of elements in
the Array: ");
 scanf("%d",&n);

 printf("\nEnter the elements:\n\n");
  for(i=0 ; i<n ; i++)
 {
  printf(" Array[%d] = ",i);
  scanf("%d",&arr[i]);
 }
```

```
for(i=0 ; i<n-1 ; i++)
 {
  for(j=0 ; j<n-i-1 ; j++)
  {
   if(arr[j]>arr[j+1]) //Swapping Condition is
Checked
   {
    temp=arr[j];
    arr[j]=arr[j+1];
    arr[j+1]=temp;
   }
  }
 }

 printf("\nThe Sorted Array is:\n\n");
 for(i=0 ; i<n ; i++)
 {
  printf(" %d",arr[i]);
 }
}
```

Complexity

What is the time complexity of the bubble sort algorithm?

For 6 elements no of swaps/comparisons in worst case 5+4+3+2+1

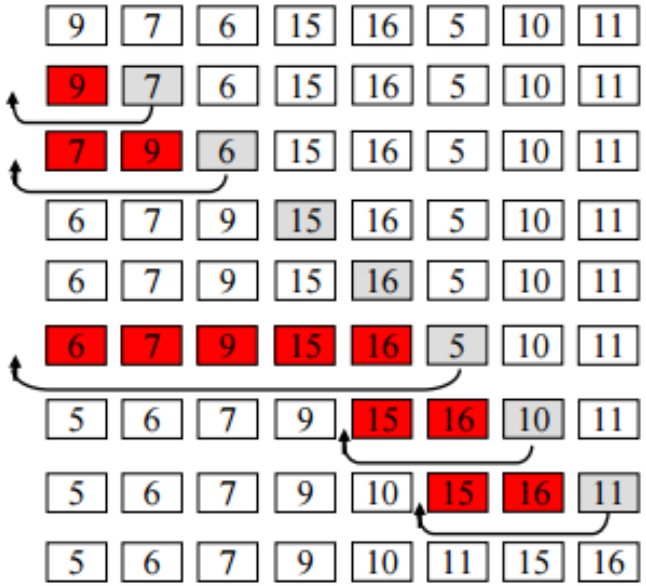
For N elements = $N*(N+1)/2 \approx N^2$

Complexity : $O(N^2)$

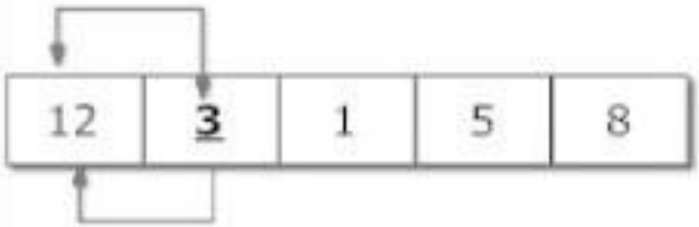
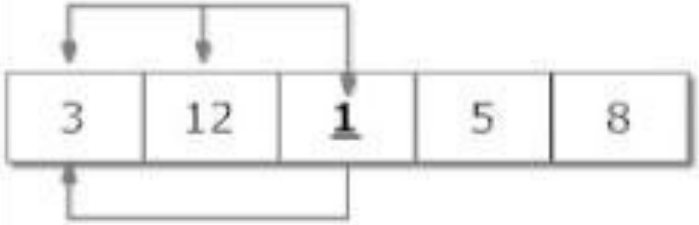
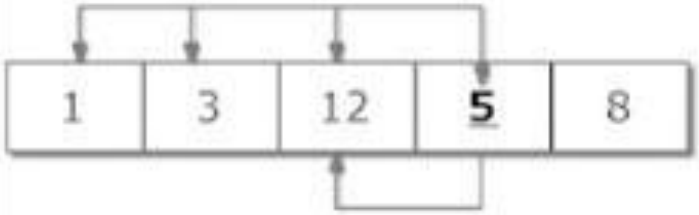
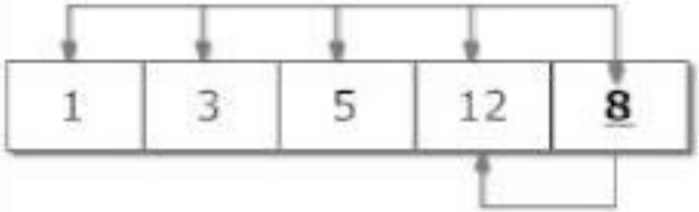
Insertion Sort

- In the insertion sort algorithm, sort an unsorted array by inserting its elements to the proper positions in an sorted array.

- The lower part of an array is maintained to be sorted. An element which is to be inserted in this sorted sub-list, has to find its appropriate place and then it is to be inserted there. Hence the name insertion sort.



Insertion Sort

Step 1		Checking second element of array with element before it and inserting it in proper position. In this case, 3 is inserted in position of 12.
Step 2		Checking third element of array with elements before it and inserting it in proper position. In this case, 1 is inserted in position of 3.
Step 3		Checking fourth element of array with elements before it and inserting it in proper position. In this case, 5 is inserted in position of 12.
Step 4		Checking fifth element of array with elements before it and inserting it in proper position. In this case, 8 is inserted in position of 12.
		Sorted Array in Ascending Order

Insertion Sort

Algorithm/Pseudo-Code:

```
for i ← 1 to i ← length(A)-1
 {
  //The values in A[ i ] are checked in-order, starting at the second one
  // save A[i] to make a hole that will move as elements are shifted
  // the value being checked will be inserted into the hole's final position
  valueToInsert ← A[i]
  holePos ← i
  // keep moving the hole down until the value being checked is larger than
  // what's just below the hole <!-- until A[holePos - 1] is <= item -->
  while holePos > 0 and valueToInsert < A[holePos - 1]
   { //value to insert doesn't belong where the hole currently is, so shift
     A[holePos] ← A[holePos - 1] //shift the larger value up
     holePos ← holePos - 1      //move the hole position down
   }
  // hole is in the right position, so put value being checked into the hole
  A[holePos] ← valueToInsert
 }
```

Insertion Sort

```
void main()
{
 int arr[20],i,n,j,k;

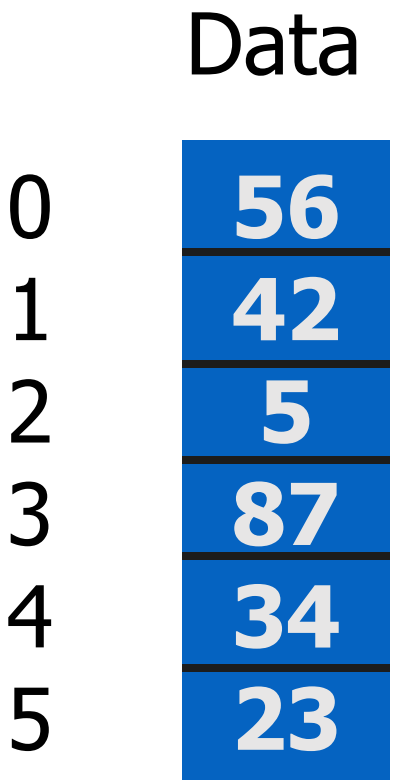
printf("\nEnter the number of elements in
the array: ");
 scanf("%d",&n);

 printf("\nEnter the elements of the
array");
 for(i=0 ; i < n ; i++)
 {
  printf("\n arr[%d] = ",i);
  scanf("%d",&arr[i]);
 }
int temp;
 for(k=1 ; k < n ; k++)
 {
  temp=arr[k];
  j=k-1;
 while((temp < arr[j]) && (j>=0))
  {
   arr[j+1]=arr[j];
   j--;
  }
  arr[j+1]=temp;
 }

printf("\nThe sorted array is: \n");
 for(i=0;i<n;i++)
 printf("%d\t",arr[i]);
}
```

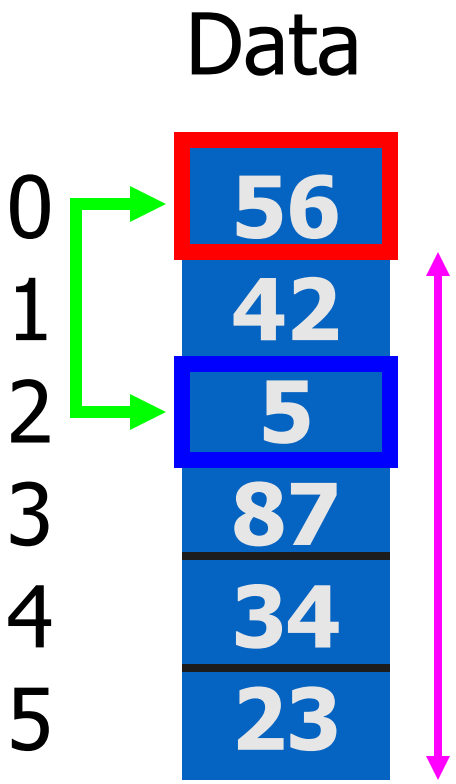
Selection Sort

- Selection sort is one of many sorting techniques available.



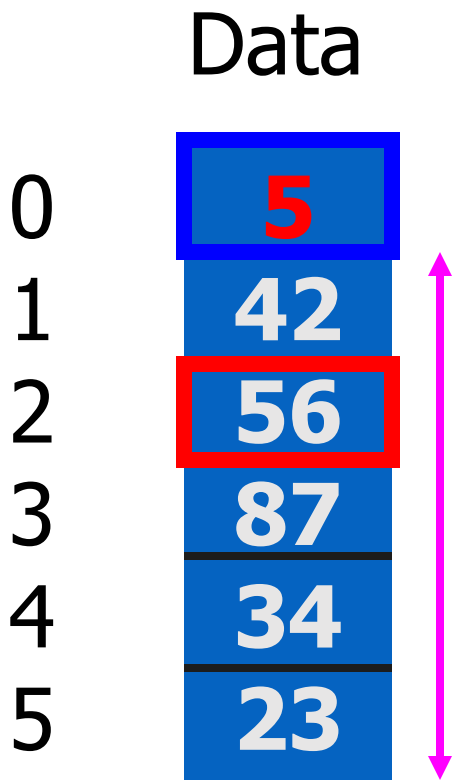
Selection Sort

- Find Smallest number and exchange with Data[0].



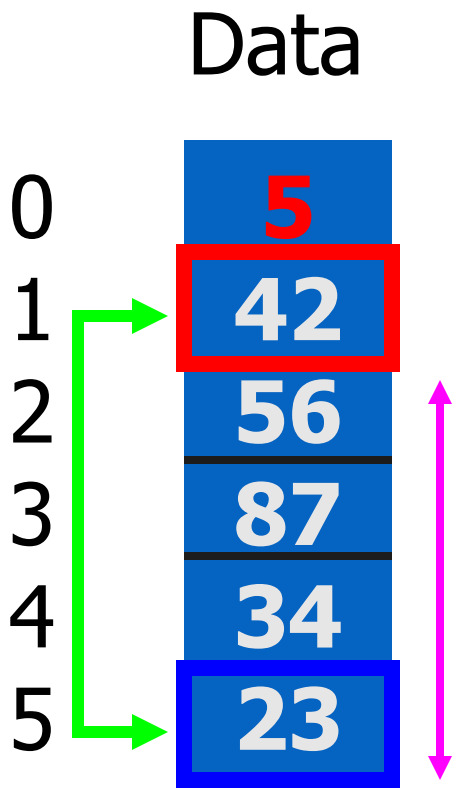
Selection Sort

- Find Smallest number and exchange with Data[0].



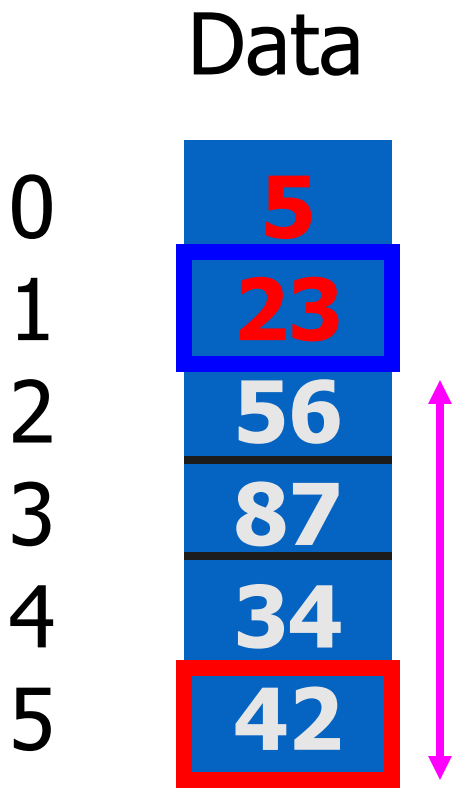
Selection Sort

- Find Smallest number and exchange with Data[1].



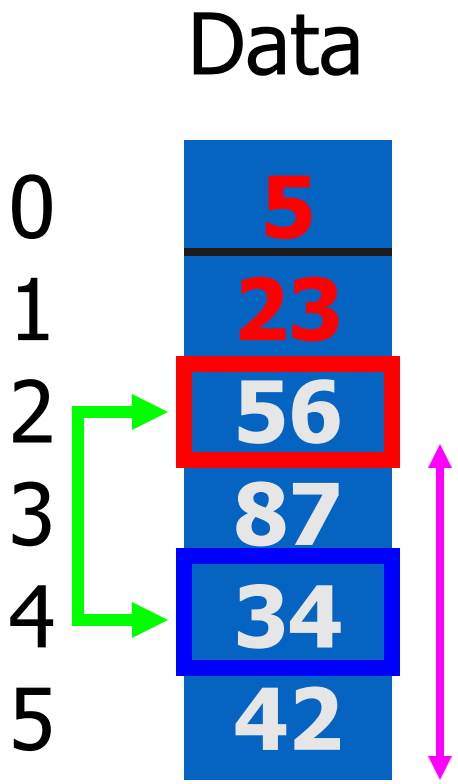
Selection Sort

- Find Smallest number and exchange with Data[1].



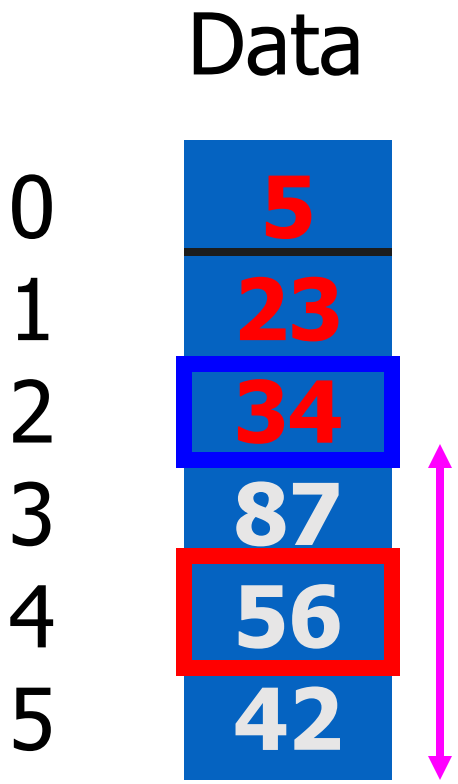
Selection Sort

- Find Smallest number and exchange with Data[2].



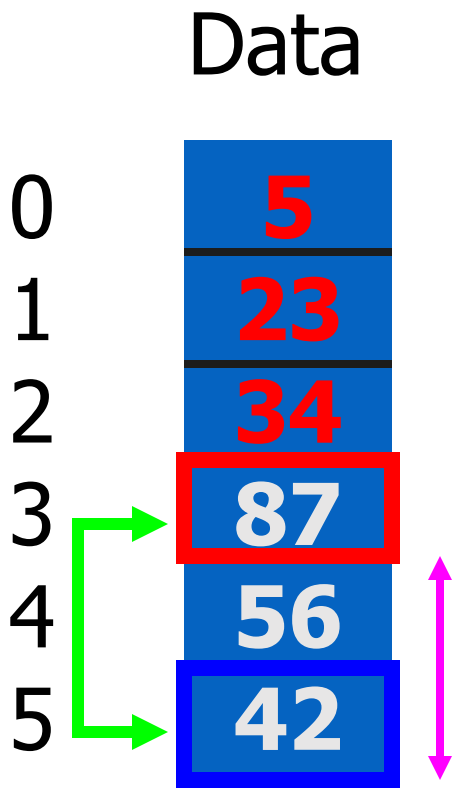
Selection Sort

- Find Smallest number and exchange with Data[2].



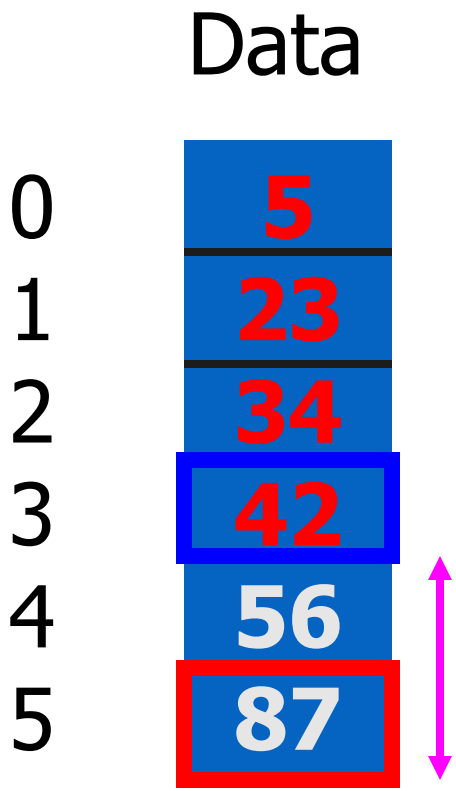
Selection Sort

- Find Smallest number and exchange with Data[3].



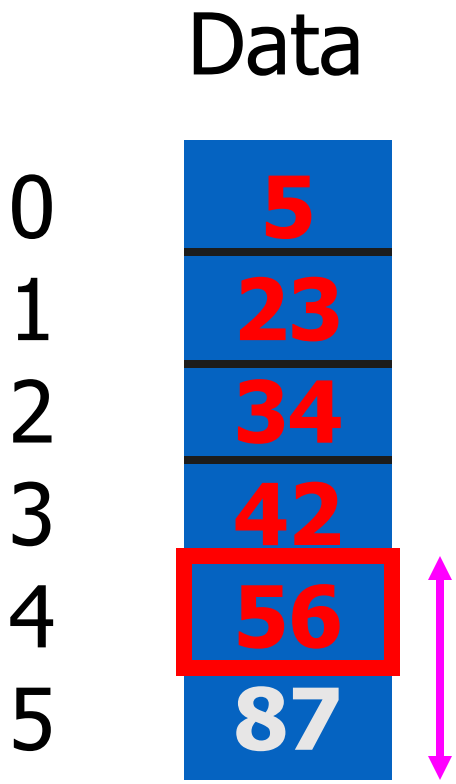
Selection Sort

- Find Smallest number and exchange with Data[3].



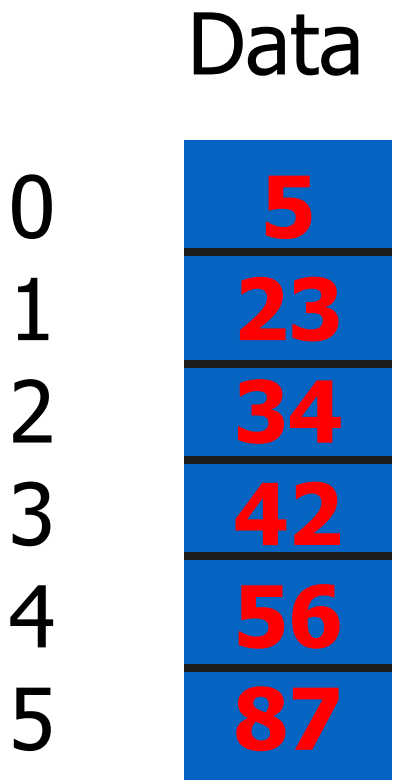
Selection Sort

- Find Smallest number and exchange with Data[4].



Selection Sort

- Sorted Data



Complexity

What is the time complexity of the Selection Sort algorithm?

For 6 elements no of swaps = 5

No of comparisons = 5+4+3+2+1

For N elements = $N*(N+1)/2 \approx N^2$

Complexity : $O(N^2)$ – How ever faster than bubble sort because no of swaps smaller.

Can We Do Better Than n^2?

Sure we can!
Selection, insertion, bubble sorts are all proportional to n^2
Other sorts are proportional to *n log n*

- Mergesort
- Quicksort
- etc.

log n is considerably smaller than n, especially as n gets larger

As the size of our problem grows, the time to run a n^2 sort will grow much faster than a *n log n* one.

END